Architecture Proposal: Interactive Conservation Map

Date: 2026-04-15 **Status:** Draft -- Open questions resolved 2026-04-15; ready for BA/PM review **Context:** Gate 07 feature addition; ECA Workshop demo deadline June 1-5, 2026

---

1. System Overview

The Interactive Conservation Map is a public-facing (Tier 1) page within the Next.js frontend that displays species collection localities, watershed boundaries, and protected areas on a Leaflet map of Madagascar. It serves as the spatial "front door" for the platform, allowing workshop attendees, researchers, and the public to visually explore where each species has been recorded, which drainage basins they inhabit, and how those locations relate to protected areas. At MVP, all displayed data is public with coordinates pre-generalized for sensitive species. The map is a new deliverable within Gate 07, sourcing its data from new models introduced at Gate 02 and new API endpoints at Gate 05.

---

2. Spatial Data Model

2.1 New Model: `species.SpeciesLocality`

This is the primary point layer for the map. Each record represents a single known locality for a species -- sourced from type descriptions, museum collection records, published literature, field observations, or eDNA surveys.

Field	Type	Notes
`id`	AutoField	PK
`species`	FK → Species	`on_delete=CASCADE`; indexed
`locality_name`	CharField(300)	e.g., "Amboaboa River at Antsirabe confluence"
`location`	PointField(srid=4326)	Exact coordinates (WGS 84). Restricted to Tier 3+ post-MVP.
`location_generalized`	PointField(srid=4326), nullable	Pre-computed generalized coordinates for Tier 1 display. Null if species is not sensitive.
`water_body`	CharField(200), blank	Name of river, lake, etc.
`water_body_type`	CharField, enum	`river` / `lake` / `stream` / `cave_system` / `wetland` / `estuary`
`drainage_basin`	FK → Watershed, nullable	See Section 2.2. FK enables efficient "all species in this watershed" queries.
`drainage_basin_name`	CharField(200), blank	Denormalized basin name for display and CSV round-tripping. Populated automatically from FK on save if FK is set.
`locality_type`	CharField, enum	`type_locality` / `collection_record` / `literature_record` / `observation`
`presence_status`	CharField, enum	`present` / `historically_present_extirpated` / `presence_unknown` / `reintroduced`

`source_citation`	TextField	Required. Provenance is non-negotiable for biodiversity data.
`year_collected`	IntegerField, nullable	Year of the record, not the publication year.
`collector`	CharField(200), blank	Collector or observer name(s).
`coordinate_precision`	CharField, enum	`exact` / `approximate` / `locality_centroid` / `water_body_centroid` — records how precise the original coordinates are. Critical for data quality assessment.
`is_sensitive`	BooleanField, default False	If True, `location_generalized` is served to Tier 1-2 users instead of `location`.
`notes`	TextField, blank	
`created_at`	DateTimeField, auto_now_add	
`updated_at`	DateTimeField, auto_now	

Design decisions:

1. **`drainage_basin` as FK (not text):** A FK to a Watershed model is strongly recommended. The primary use case "show all species in the Betsiboka watershed" becomes a simple JOIN (`SpeciesLocality.objects.filter(drainage_basin_id=X)`) rather than a spatial intersection query (`ST_Contains(watershed.geom, locality.location)`). The FK approach is faster by orders of magnitude, simpler to implement, and does not require the frontend to perform spatial queries. The denormalized `drainage_basin_name` field preserves CSV round-trip simplicity and display convenience.

2. **Uniqueness constraint:** `unique_together = (species, location, locality_type)` . This prevents exact duplicates from the same source type while allowing the same coordinates to appear with different locality types (e.g., a type locality that is also a recent observation). Different source citations at the same location and type should be merged into a single record with combined citations in `source_citation` , not stored as separate rows. This is cleaner than allowing unlimited duplicates.

3. **`coordinate_precision` field:** Added to the stakeholder specification. Many historical locality records are geocoded from textual locality descriptions ("Antsirabe region") with varying precision. Without tracking precision, the map would misleadingly present all points as equally authoritative. This field is essential for honest data representation.

4. **`is_sensitive` vs. species-level sensitivity:** Sensitivity is set per-locality rather than per-species because some localities for a given species may be public knowledge (published type localities) while others are sensitive (newly discovered populations of the same species). A species-level flag is too coarse. At MVP, the seed data curator sets `is_sensitive` manually per record. Post-MVP, a rule engine can auto-flag based on IUCN status and locality recency.

5. **`location_generalized` as pre-computed field:** Generalization is pre-computed at data entry time (via a model `save()` override or management command) rather than computed on-the-fly at the API layer. Rationale: generalization to a 0.1-degree grid cell is a lossy, deterministic operation

that should produce the same result every time. Pre-computing avoids repeating the calculation on every API request and ensures the generalized coordinate is reviewable by data curators in Django Admin.

Generalization algorithm (for `save()` override):

```
If is_sensitive and location is not null:
    location_generalized = Point(
        round(location.x, 1),  # longitude to 0.1 degree (~11km at equator, ~9km at
Madagascar's latitude)
        round(location.y, 1),  # latitude to 0.1 degree
    )
```

This follows GBIF's recommended approach for sensitive species: snap to a 0.1-degree grid. At Madagascar's latitude (~12-25 S), 0.1 degree longitude is approximately 9.7-10.5 km.

2.2 New Model: `species.Watershed`

Reference layer storing HydroSHEDS HydroBASINS polygons for Madagascar.

Field	Type	Notes
`id`	AutoField	PK
`hybas_id`	BigIntegerField, unique	HydroBASINS feature ID from source data
`name`	CharField(200)	Basin name; many HydroBASINS features lack names -- use "Unnamed basin [hybas_id]" as fallback
`pfafstetter_level`	IntegerField	Pfafstetter coding level (1-12). MVP loads level 6 only.
`pfafstetter_code`	BigIntegerField	Pfafstetter basin code
`parent_basin`	FK → self, nullable	For hierarchical basin navigation (parent = level N-1)
`area_sq_km`	DecimalField(12,2), nullable	Basin area from HydroBASINS attributes
`geometry`	MultiPolygonField(srid=4326)	Basin boundary polygon
`created_at`	DateTimeField, auto_now_add	

Pfafstetter level recommendation: Level 6 for MVP. Rationale:

- Level 4 (~10 basins for Madagascar): Too coarse. Major basins are too large to be ecologically meaningful for freshwater fish that may be restricted to single tributaries.
- Level 6 (~60-100 basins for Madagascar): Good balance. Basins correspond roughly to major river systems and their sub-catchments. Manageable polygon count for frontend rendering. Ecologically meaningful groupings for freshwater fish distribution.
- Level 8 (~500+ basins): Too fine for MVP. Increases data volume and rendering complexity without proportional benefit at the "overview map" zoom level.

- Post-MVP, loading additional levels (4, 8) enables drill-down from major basins to sub-catchments.

2.3 New Model: `species.ProtectedArea`

Reference layer storing WDPA protected area polygons for Madagascar.

Field	Type	Notes
`id`	AutoField	PK
`wdpa_id`	IntegerField, unique	WDPA feature ID
`name`	CharField(300)	Protected area name
`designation`	CharField(200)	e.g., "National Park", "Special Reserve", "Community Reserve"
`iucn_category`	CharField(20), blank	IUCN PA category (Ia, Ib, II, III, IV, V, VI)
`status`	CharField(100)	e.g., "Designated", "Proposed"
`status_year`	IntegerField, nullable	Year of designation
`area_km2`	DecimalField(12,2), nullable	Reported area
`geometry`	MultiPolygonField(srid=4326)	PA boundary
`created_at`	DateTimeField, auto_now_add	

---

3. Reference Layer Storage Strategy

Recommendation: Hybrid approach (PostGIS tables + cached GeoJSON API responses).

Concern	PostGIS Tables	Static GeoJSON Files	Hybrid (Recommended)
Spatial queries ("species in basin X")	Native `ST_Contains` / `ST_Intersects`	Not possible server-side	PostGIS handles queries
Frontend rendering	Serialize to GeoJSON per request	Serve static files directly	Serialize once, cache in Redis or filesystem
Data volume control	Filter in SQL (only Madagascar)	Must pre-filter files	Filter at load time, cache result
Admin editability	Full Django Admin CRUD	Requires file replacement	Admin for metadata; geometry via data loads
Complexity	Models + migrations + load scripts	Simplest	Moderate -- but justified by query need

Why not static GeoJSON only: The `drainage_basin` FK on SpeciesLocality already commits us to a PostGIS Watershed table. Static files would create a parallel data path that must be kept in sync. Additionally, the "all species in this watershed" query is a core use case that requires a database JOIN.

Why not PostGIS-only (no caching): Serializing ~60-100 watershed polygons or ~100+ protected area polygons to GeoJSON on every page load is wasteful. These reference geometries change at most annually. A cached GeoJSON response (Redis with 24-hour TTL, or a pre-generated static file rebuilt by management command) eliminates redundant serialization.

Implementation:

1. Watershed and ProtectedArea polygons stored in PostGIS tables.
2. A management command (`generate_map_layers`) serializes each layer to GeoJSON and writes to `staticfiles/map-layers/watersheds.geojson` and `staticfiles/map-layers/protected-areas.geojson` . These are served as static files by the CDN/webserver.
3. The DRF API serves species locality points dynamically (they change with data entry). Reference layers are served as static files (they change at most annually).
4. When reference data is re-loaded (e.g., annual WDPA update), the management command is re-run to regenerate the static GeoJSON files.

3.1 Data Source Specifications

HydroSHEDS HydroBASINS:

- Source: https://www.hydrosheds.org/products/hydrobasins
- License: Free for non-commercial and commercial use with attribution (CC BY 4.0 equivalent for HydroSHEDS v2; check current license terms)
- File: `hybas_af_lev06_v1c.shp` (Africa, level 6) -- filter to Madagascar by bounding box or country intersection
- Estimated Madagascar subset: ~60-100 polygons at level 6
- File size estimate: ~2-5 MB as GeoJSON after Madagascar filtering and geometry simplification

WDPA (World Database on Protected Areas):

- Source: https://www.protectedplanet.net/en/thematic-areas/wdpa
- License: Free for non-commercial use; requires attribution. Commercial use requires IBAT subscription. This platform qualifies as non-commercial.
- Download: Monthly updated shapefile or GeoPackage. Filter to `ISO3 = 'MDG'` .
- Estimated Madagascar subset: ~120-160 protected areas (Madagascar has an extensive PA network covering ~10% of land area)
- File size estimate: ~5-15 MB as GeoJSON after simplification. Some large PAs (e.g., Masoala NP) have complex coastline boundaries.

Geometry simplification: Both datasets should be simplified at load time using PostGIS `ST_Simplify` with a tolerance of ~~0.001 degrees (~~100m). This reduces vertex counts by 70-90% with negligible visual impact at the zoom levels used for a country-level overview map. The management command that generates the static GeoJSON files applies this simplification.

---

4. API Endpoint Design

4.1 Species Localities GeoJSON Endpoint

```
GET /api/v1/map/localities/
```

Returns a GeoJSON FeatureCollection of species localities. This is the only dynamic map endpoint; reference layers are served as static files.

Query parameters:

Parameter	Type	Description
`species_id`	integer	Filter to a single species
`family`	string	Filter by family name (e.g., `Bedotiidae`)
`iucn_status`	string	Filter by IUCN category (e.g., `CR`, `EN`)
`watershed_id`	integer	Filter by Watershed FK
`locality_type`	string	Filter by locality type enum
`presence_status`	string	Filter by presence status enum
`water_body_type`	string	Filter by water body type enum
`bbox`	string	Bounding box filter: `min_lng,min_lat,max_lng,max_lat`

Response (200 OK):

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [47.52, -18.91]
      },
      "properties": {
        "id": 42,
        "species_id": 7,
        "scientific_name": "Pachypanchax sakaramyi",
        "family": "Aplocheilidae",
        "iucn_status": "EN",
        "locality_name": "Sakaramy River near Joffreville",
        "locality_type": "type_locality",
        "presence_status": "present",
        "water_body": "Sakaramy River",
        "water_body_type": "river",
        "drainage_basin_name": "Sambirano",
        "year_collected": 1928,
        "coordinate_precision": "exact",
        "source_citation": "Holly, 1928"
      }
    }
  ]
}
```

Tier behavior at MVP: All requests are unauthenticated (Tier 1). The endpoint serves `location_generalized` coordinates for records where `is_sensitive = True`, and `location` for

non-sensitive records. The `geometry` field in the GeoJSON response is always the appropriate coordinate for the requester's tier. At MVP, this logic is:

```
def get_display_location(self, obj):
    if obj.is_sensitive and obj.location_generalized:
        return obj.location_generalized
    return obj.location
```

Post-MVP, when authenticated map access is implemented, this becomes tier-aware:

```
def get_display_location(self, obj, request):
    if obj.is_sensitive and request.user.access_tier < 3:
        return obj.location_generalized
    return obj.location
```

Performance considerations:

- Expected data volume: ~300-800 locality points across ~79 species (estimated 4-10 localities per species on average). This is small enough to return all points in a single response without server-side clustering or tile-based loading.
- Use `select_related('species', 'drainage_basin')` to avoid N+1 queries.
- Apply `only()` to limit fields fetched from the database to those needed for GeoJSON serialization.
- Cache the full unfiltered GeoJSON response in Redis (5-minute TTL). Invalidate on SpeciesLocality save/delete signals. Filtered responses are not cached at MVP (data volume is too small to warrant per-filter caching).

4.2 Watershed List Endpoint

```
GET /api/v1/map/watersheds/
```

Returns a JSON list of watersheds (without geometry) for populating filter dropdowns and info panels.

Response (200 OK):

```
[
  {
    "id": 1,
    "name": "Betsiboka",
    "pfafstetter_level": 6,
    "area_sq_km": 48900.00,
    "species_count": 12
  }
]
```

`species_count` is annotated via a subquery on SpeciesLocality. Cached with the static layer files.

Watershed and protected area GeoJSON geometries are served as static files, not through DRF:

```
GET /static/map-layers/watersheds.geojson
GET /static/map-layers/protected-areas.geojson
```

4.3 Map Summary Endpoint

```
GET /api/v1/map/summary/
```

Returns aggregate statistics for the map page header/sidebar.

Response (200 OK):

```
{
  "total_localities": 547,
  "species_with_localities": 72,
  "species_without_localities": 7,
  "watersheds_represented": 34,
  "locality_type_counts": {
    "type_locality": 79,
    "collection_record": 312,
    "literature_record": 98,
    "observation": 58
  },
  "presence_status_counts": {
    "present": 420,
    "historically_present_extirpated": 67,
    "presence_unknown": 45,
    "reintroduced": 15
  }
}
```

Cached in Redis (5-minute TTL), invalidated on SpeciesLocality changes.

4.4 Endpoint Design Rationale

Why separate `/api/v1/map/` namespace instead of `?format=geojson` on `/api/v1/species/`:

1. The species list endpoint returns species-centric data (one row per species). The map endpoint returns locality-centric data (multiple rows per species, with geometry). These are fundamentally different response shapes.
2. A `format=geojson` parameter on the species endpoint would require the species serializer to conditionally reshape its entire output, violating the principle of one endpoint = one response contract.
3. Separate endpoints allow independent caching strategies (species list cached by page; map GeoJSON cached as a single blob).
4. The map endpoints will eventually serve authenticated users with different coordinate precision -- this is cleaner to implement on a dedicated endpoint than as a format variant.

---

5. Estimated Data Volumes

Dataset	Record Count	GeoJSON Size (est.)	Notes
Species localities	300-800 points	100-300 KB	~4-10 localities per species. Some well-studied species (e.g., *Paretroplus menarambo*)

			may have 20+ records.
Watershed polygons (level 6)	60-100 polygons	2-5 MB (simplified)	After ST_Simplify at 0.001 degree tolerance
Protected area polygons	120-160 polygons	5-15 MB (simplified)	Some PAs have complex boundaries; simplification critical
Total initial map payload		**7-20 MB**	Loaded on demand per layer toggle, not all at once

Frontend loading strategy: Reference layer GeoJSON files are loaded lazily when the user toggles a layer on. Locality points are loaded on initial map render (they are the primary layer). This means initial page load fetches ~100-300 KB of locality data plus the Leaflet tile base map. Watershed and PA layers add 2-15 MB each only when toggled.

---

6. Gate Impact Assessment

The conservation map feature touches five existing gates. Below is the recommended change to each gate, sequenced to minimize disruption.

Gate 02 Additions (Data Layer)

Add the following models to Gate 02:

App	New Models
`species`	`SpeciesLocality`, `Watershed`, `ProtectedArea`

Rationale: These are data-layer models with no business logic, consistent with Gate 02's scope ("just models and migrations"). Adding them to Gate 02 means migrations are complete before any dependent gate runs.

PostGIS geometry fields: Gate 02 currently states "No spatial data in MVP models" in its Out of Scope section. This must be revised. PostGIS is already configured (Gate 01 enables `django.contrib.gis`), so adding geometry fields is mechanically simple -- it is the scope decision that changes, not the infrastructure.

Specific changes to Gate 02:

- Add `SpeciesLocality`, `Watershed`, `ProtectedArea` to the `species` app models table
- Add full field specifications (as in Section 2 above) to the Model Specifications section
- Remove "PostGIS geometry fields -- no spatial data in MVP models" from Out of Scope
- Add acceptance criteria for spatial model validation (PointField accepts valid coordinates, MultiPolygonField accepts valid polygons, unique constraints on `hybas_id` and `wdpa_id`)

Gate 04 Additions (Django Admin)

Add admin registrations for the three new models:

`species.SpeciesLocality`:

```
list_display = [
    "species", "locality_name", "locality_type", "presence_status",
    "water_body", "drainage_basin", "year_collected", "coordinate_precision",
]
list_filter = [
    "locality_type", "presence_status", "water_body_type",
    "coordinate_precision", "is_sensitive", "drainage_basin",
]
search_fields = [
    "locality_name", "water_body", "species__scientific_name",
    "source_citation", "collector",
]
readonly_fields = ["location_generalized", "created_at", "updated_at"]
raw_id_fields = ["species", "drainage_basin"]
```

Note: `location_generalized` is read-only because it is auto-computed from `location` + `is_sensitive` on save. Django Admin's default `OSMGeoAdmin` widget provides a map-based point picker for the `location` field, which is valuable for data curators verifying coordinates.

`species.Watershed`:

```
list_display = ["name", "pfafstetter_level", "pfafstetter_code", "area_sq_km"]
list_filter = ["pfafstetter_level"]
search_fields = ["name"]
readonly_fields = ["hybas_id", "geometry", "created_at"]  # geometry loaded via
management command, not admin
```

`species.ProtectedArea`:

```
list_display = ["name", "designation", "iucn_category", "status", "area_km2"]
list_filter = ["designation", "iucn_category", "status"]
search_fields = ["name"]
readonly_fields = ["wdpa_id", "geometry", "created_at"]  # geometry loaded via
management command, not admin
```

Bulk import support: SpeciesLocality records will primarily be loaded via the `seed_localities` management command (Gate 06), not through Django Admin one-by-one. However, Django Admin must support individual record creation and editing for ongoing data maintenance. The admin `OSMGeoAdmin` widget for the `location` field allows point-and-click coordinate entry, which is valuable for adding new locality records discovered in literature.

Gate 05 Additions (DRF API)

Add three new endpoints to the endpoint table:

Method	Path	Min Tier	Description
`GET`	`/api/v1/map/localities/`	1 (public)	Species localities as GeoJSON FeatureCollection
`GET`	`/api/v1/map/watersheds/`	1 (public)	Watershed list (no geometry; for filters/info)

`GET`	`/api/v1/map/summary/`	1 (public)	Map aggregate statistics

New serializers:

- `SpeciesLocalityGeoSerializer` — GeoJSON Feature serializer using `rest_framework_gis.serializers.GeoFeatureModelSerializer`. Serves `location` or `location_generalized` based on `is_sensitive` flag (and post-MVP, request tier).
- `WatershedListSerializer` — standard DRF serializer with annotated `species_count`.
- `MapSummarySerializer` — read-only serializer for aggregate stats.

Dependency: `djangorestframework-gis` must be added to requirements. This is the standard DRF extension for GeoJSON serialization and is well-maintained.

Gate 06 Additions (Seed Data)

Add two new management commands and one new seed data file:

1. **`seed_localities` management command:**

   ```
   python manage.py seed_localities --csv
   data/seed/madagascar_freshwater_fish_localities_seed.csv [--dry-run]
   ```

   - Idempotent: keyed on `(species__scientific_name, location, locality_type)`
   - Validates coordinates are within Madagascar bounding box (43.0-51.0 E, -26.0 to -11.5 S)
   - Auto-assigns `drainage_basin` FK by performing `ST_Contains` spatial query against loaded Watershed polygons (requires watersheds to be loaded first)
   - Auto-computes `location_generalized` for records with `is_sensitive = True`
   - Logs: N created, N updated, N skipped, with error details

2. **`load_reference_layers` management command:**

   ```
   python manage.py load_reference_layers --watersheds
   data/reference/hydrobasins_madagascar_lev06.shp --protected-areas
   data/reference/wdpa_madagascar.shp [--simplify 0.001]
   ```

   - Loads HydroBASINS and WDPA shapefiles into Watershed and ProtectedArea tables
   - Applies geometry simplification (`ST_Simplify`) at specified tolerance
   - Idempotent: keyed on `hybas_id` and `wdpa_id`
   - Must run before `seed_localities` (localities need watershed polygons for FK assignment)

3. **`generate_map_layers` management command:**

   ```
   python manage.py generate_map_layers --output-dir staticfiles/map-layers/
   ```

   - Serializes Watershed and ProtectedArea tables to GeoJSON static files
   - Applies additional simplification for frontend rendering if needed
   - Run after `load_reference_layers` and after any reference data updates

Execution order for full seed:

```
python manage.py load_reference_layers --watersheds ... --protected-areas ...
python manage.py seed_species --csv data/seed/madagascar_freshwater_fish_seed.csv
```

```
python manage.py seed_localities --csv
data/seed/madagascar_freshwater_fish_localities_seed.csv
python manage.py generate_map_layers --output-dir staticfiles/map-layers/
```

Gate 07 Additions (Public Frontend)

Add one new page to Gate 07 deliverables:

Page: `/map/` **Rendering:** CSR (client-side rendering). Map interactions (pan, zoom, layer toggle, click) are inherently client-side; SSR provides no benefit for a map page.

UI sections:

1. **Full-viewport Leaflet map** with OpenStreetMap tile base layer. Centered on Madagascar (-18.9, 47.5), zoom level 6.
2. **Layer toggle panel** (top-right or sidebar):
   - "Species Localities" (on by default) -- point markers
   - "Watersheds" (off by default) -- polygon overlay
   - "Protected Areas" (off by default) -- polygon overlay
3. **Filter panel** (sidebar or collapsible):
   - Family dropdown (populated from API)
   - IUCN status multi-select
   - Locality type multi-select
   - Presence status multi-select
   - Watershed dropdown (populated from `/api/v1/map/watersheds/`)
4. **Locality markers:**
   - Color-coded by IUCN status (standard IUCN colors: CR=red, EN=orange, VU=yellow, etc.)
   - Icon shape or border varies by presence status (solid=present, hollow=extirpated, dashed=unknown)
   - Click opens popup with: species name (linked to `/species/{id}/`), IUCN status badge, locality name, locality type, presence status, water body, year, source citation
   - Multiple localities at the same coordinates use Leaflet.markercluster for decluttering
5. **Map statistics bar** (top or bottom): Total localities shown / total species represented (updates with filters).
6. **Legend:** Color key for IUCN status; shape key for presence status.

Frontend dependencies:

- `leaflet` + `react-leaflet` (React wrapper for Leaflet)
- `leaflet.markercluster` (marker clustering plugin)
- No additional mapping libraries. Leaflet is specified in CLAUDE.md.

Estimated frontend effort: The map page is the most complex single page in Gate 07 due to Leaflet integration, layer management, and interactive filtering. Estimate 2-3 of the available ~7 weeks. This is feasible alongside the other three Gate 07 pages (species directory, species profile, dashboard) if the map page uses the same component library (Tailwind) and design system.

---

7. Recommended CSV Schema

File: `data/seed/madagascar_freshwater_fish_localities_seed.csv`

Column	Maps to Field	Required	Notes
`scientific_name`	FK lookup → Species	Yes	Must match `Species.scientific_name` exactl
`locality_name`	`locality_name`	Yes	Free text
`latitude`	`location` (y)	Yes	Decimal degrees, WGS 84. Negative for Sc
`longitude`	`location` (x)	Yes	Decimal degrees, WGS 84. Positive for Eas
`water_body`	`water_body`	No	River/lake name
`water_body_type`	`water_body_type`	No	Enum: `river/lake/stream/cave_system/wetlan`
`locality_type`	`locality_type`	Yes	Enum: `type_locality/collection_record/lit`
`presence_status`	`presence_status`	Yes	Enum: `present/historically_present_extirpated/pr`
`coordinate_precision`	`coordinate_precision`	Yes	Enum: `exact/approximate/locality_centroid/`
`source_citation`	`source_citation`	Yes	Full citation or short reference
`year_collected`	`year_collected`	No	Integer year
`collector`	`collector`	No	Name(s)
`is_sensitive`	`is_sensitive`	No	`true/false`. Default: `false`.
`notes`	`notes`	No	Free text

Not in CSV (auto-computed):

- `drainage_basin` -- assigned by spatial query during import
- `drainage_basin_name` -- populated from FK
- `location_generalized` -- computed from `location` + `is_sensitive`

Validation rules applied by `seed_localities`:

- Latitude must be between -26.0 and -11.5 (Madagascar extent)
- Longitude must be between 43.0 and 51.0 (Madagascar extent)
- `scientific_name` must match an existing Species record (skip row with warning if not found)
- Enum fields must match allowed values (skip row with warning if invalid)
- `source_citation` must not be empty

---

8. Locality Data Sourcing Outline

File: `docs/data-sources/locality-data-sourcing.md`

This document should be written by the data curator (project lead) to accompany the localities CSV. Below is the recommended structure and content outline.

1. Purpose and Scope

- This document describes the provenance, methodology, and known limitations of the species locality dataset used to seed the conservation map.
- Covers ~79 endemic species with an expected 300-800 locality records.

2. Primary Data Sources

Source	Type	Coverage	Access
Leiss et al. 2022 (Zoo Biology 41:244-262)	Published paper + supplementary data	Baseline species list; institutional holdings; general distribution information	Published; open access
GBIF occurrence records	Digitized museum specimens + field observations	Variable coverage; strongest for Bedotiidae and Cichlidae; weak for cave fish and recently described species	Free download; CC-BY or CC0 per dataset
FishBase distribution records	Compiled from published literature	General locality descriptions; often not georeferenced	Free access
Oliveira Carvalho et al. 2024 (eDNA survey)	Recent field survey	Environmental DNA detections in specific watersheds; provides locality data for species not captured by traditional methods	Published; contact authors for exact coordinates
CARES beta species list	Conservation priority list	Species names and priority status; limited locality data	CARES membership
Primary taxonomic literature	Type descriptions and revisions	Type localities for all described species; collection records from revisionary works	Published; library access
Sparks & Stiassny (various)	Systematic revisions of Bedotiidae, Cichlidae	Detailed collection localities with coordinates for many species	Published
Stiassny & Raminosoa (1994)	Checklist of freshwater fishes of Madagascar	Historical baseline; many localities described textually	Published

3. Geocoding Methodology

- Type localities from published descriptions: coordinates taken directly from publication if provided; otherwise geocoded from locality description using GeoNames + Google Earth.
- GBIF occurrences: coordinates taken directly from GBIF download; filtered to those with `coordinateUncertaintyInMeters < 10000`.
- Literature records without coordinates: geocoded to water body centroid or locality centroid; `coordinate_precision` set to `water_body_centroid` or `locality_centroid` accordingly.
- All coordinates verified against Madagascar boundary and checked for obvious errors (coordinates in the ocean, wrong hemisphere).

4. Precision Levels

- `exact` : Coordinates from GPS-recorded field collections or published with sub-km precision.
- `approximate` : Published with coarse precision (e.g., "near Antsirabe") or from older museum specimens.
- `locality_centroid` : Geocoded to the center of a named locality (village, town).
- `water_body_centroid` : Geocoded to the approximate center of a named river reach or lake.

5. Sensitivity Assessment

- Species with IUCN status CR or EN and locality records from the last 20 years: marked `is_sensitive = true` .
- Type localities from publications older than 50 years: generally not sensitive (already public knowledge).
- Cave fish (*Typhleotris* spp.): all localities marked sensitive regardless of age (extremely restricted range, high collection pressure risk).
- Sensitivity decisions are conservative at MVP; post-MVP, sensitivity will be managed per-locality via Django Admin.

6. Known Gaps

- Several recently described species (post-2020) may have only the type locality and no additional records.
- Cave fish locality data is sparse and often imprecise.
- Eastern rainforest species (Bedotiidae) have patchy coverage -- many streams have never been surveyed.
- Non-endemic species are excluded from the initial dataset (focus on the ~79 endemics).
- eDNA detections (Oliveira Carvalho et al. 2024) provide presence data but exact locality coordinates may require author permission.

7. Update Process and Contributor Instructions

- New locality records can be submitted via Django Admin (Tier 3+ coordinators) or by providing updated CSV rows to the data curator.
- CSV rows must include all required fields (see Section 7 of the architecture proposal).
- New sources should be cited in full in `source_citation` .
- Coordinate precision must be honestly assessed and recorded.

---

9. Risks and Tradeoffs

Technical Risks

Risk	Severity	Mitigation
GeoJSON payload size for reference layers	Medium	Protected area polygons can be large (5-15 MB). Mitigation: aggressive geometry simplification at load time; lazy loading (layers off by default); consider TopoJSON for 60-80% size reduction if GeoJSON proves too large.
Leaflet performance with many polygons	Medium	100+ watershed polygons + 150+ PA polygons rendering simultaneously could cause frame drops on low-end devices. Mitigation: layers are off by default; only one reference layer visible at a time is the expected usage pattern; simplify geometries aggressively.

Locality data quality and completeness	High	The localities CSV is the map's foundation and it does not yet exist. The data curator must compile it from multiple heterogeneous sources with varying coordinate precision. Mitigation: the `coordinate_precision` field honestly represents data quality; the `--dry-run` flag on the seed command catches errors before they enter the database.
HydroBASINS naming gaps	Low	Many HydroBASINS features at level 6 lack human-readable names. They have Pfafstetter codes but no names. Mitigation: supplement with known Malagasy river basin names from literature; use "Basin [code]" as fallback. This is a data curation task, not a technical blocker.
Coordinate generalization is one-way lossy	Low	Once generalized coordinates are served publicly, they cannot be "un-served." This is by design (same as GBIF practice) but means sensitivity decisions must be made carefully before data publication. Mitigation: `is_sensitive` defaults to `false`; curators must opt in to sensitivity per record.

Tradeoffs

Decision	Chosen	Alternative	Why
FK for drainage_basin	FK to Watershed model	Text field	FK enables efficient queries and enforces referential integrity. Costs one spatial query per locality at import time (acceptable for batch import of 300-800 records).
Pre-computed generalization	`location_generalized` stored on model	Computed at serializer level per request	Pre-computation is reviewable in Admin, avoids repeated calculation, and produces deterministic results. Costs one extra PointField per record (~16 bytes).
Static GeoJSON for reference layers	Generated files served via CDN	DRF endpoints with per-request serialization	Reference data changes annually at most. Static files are faster to serve and simpler to cache.
Separate /map/ endpoints	Dedicated namespace	Format parameter on /species/	Different data shape (locality-centric vs species-centric), different caching needs, cleaner separation of concerns.
Level 6 Pfafstetter for MVP	~60-100 basins	Level 4 (~~10) or Level 8 (~~500+)	Balances ecological relevance with rendering performance. Multi-level drill-down is a post-MVP enhancement.

Resolved Questions (2026-04-15)

1. **Localities CSV timeline.** RESOLVED: Feasible. Project lead will compile the CSV alongside development work before the June 1 deadline. The parallel implementation sequence (weeks 2-6 for data curation) accommodates this.

2. **Satellite vs. street map base tiles.** RESOLVED: Both. Offer OpenStreetMap and ESRI World Imagery (free for non-commercial use) as switchable base layers via Leaflet's layer control. OpenStreetMap is the default; satellite is the alternative. Minimal additional implementation cost.

3. **Watershed naming.** RESOLVED: Cross-reference HydroBASINS polygons with published hydrological maps of Madagascar (e.g., Chaperon et al. 1993 atlas, ORSTOM/IRD maps) to name the major basins (~15-20 well-known systems: Betsiboka, Tsiribihina, Mangoky, Onilahy, Mangoro, Sofia, etc.). Smaller sub-basins receive auto-generated names as "Sub-basin of [parent name]" using the Pfafstetter hierarchy. These can be manually renamed over time as data curation allows. The `load_reference_layers` management command should implement this naming strategy: match known basin names where available, fall back to "Sub-basin of [nearest named parent]" for the rest.

4. **TopoJSON for reference layers.** RESOLVED: Start with GeoJSON. If file sizes prove problematic during Gate 07 testing (>10 MB per layer after simplification), switch to TopoJSON at that point. The hybrid approach (static files served to frontend) supports either format without backend changes. Add `topojson-client` (~15 KB) to the frontend only if needed. No architectural decision required now — this is an optimization gate during testing.

---

10. Implementation Sequence

Given the June 1 deadline (~7 weeks), the recommended implementation order for map-related work across gates:

Week	Gate	Task
1	02	Add SpeciesLocality, Watershed, ProtectedArea models + migrations
1	06	Write `load_reference_layers` management command; download + filter HydroSHEDS and WDPA data
2	04	Add admin registrations for new models (SpeciesLocality with OSMGeoAdmin)
2	06	Write `seed_localities` management command; write `generate_map_layers` command
2-3	--	**DATA CURATION:** Project lead compiles localities CSV (parallel with development)
3	05	Add `/api/v1/map/localities/`, `/api/v1/map/watersheds/`, `/api/v1/map/summary/` endpoints
4-5	07	Build `/map/` page with Leaflet, layer toggles, filtering, popups
6	06	Load real localities CSV; generate static map layer files; verify on staging
7	07	Polish, test, deploy to staging for ECA Workshop

This sequence parallelizes backend model/API work (weeks 1-3) with data curation (weeks 2-6) and frontend development (weeks 4-7). The critical path is the localities CSV -- if it is delayed, the map page

can be demonstrated with reference layers and a partial dataset.